

UD02: Control de flujo en los programas

Contenido

Introducción	2
1. Estructuras de selección	2
1.2 Sentencia if-else	2
1.3 Sentencia if-elif-else	3
1.4 Operadores lógicos y de comparación	4
1.5 Sentencia match-case	4
2. Estructuras de repetición (Bucles)	7
2.1 Bucle while	7
2.2 Bucle for	7
3. Estructuras de salto en bucles	9
3.1 Sentencia break	9
3.2 Sentencia continue	9
3.3 Sentencia pass	10
4. Control de excepciones	11
4.1 ¿Qué es una excepción?	11
4.2 Manejo de excepciones con try-except	11
4.3 Bloques else y finally	12
4.4 Lanzar excepciones con raise	12
4.5 Creación de excepciones personalizadas	13
5. Aserciones	14
5.1 ¿Qué es una aserción?	14
6. Prueba, depuración y documentación de la aplicación	15
6.1 Pruebas de programas	15
6.2 Depuración de código	15
6.3 Documentación del código	15

Introducción

El **control de flujo** es la capacidad de un programa para tomar decisiones, repetir acciones y gestionar situaciones inesperadas. En Python, esto se consigue principalmente con estructuras de selección, estructuras de repetición y manejo de excepciones.

1. Estructuras de selección

Las estructuras de selección te permiten ejecutar diferentes bloques de código en función del cumplimiento de una o varias condiciones. Son el pilar para que un programa pueda "pensar" y tomar decisiones.

1.1 Sentencia if

La sentencia if es la más básica y fundamental. Ejecuta un bloque de código **si** la condición es True. Si la condición es False, el bloque se ignora por completo.

Ejemplo:

```
edad = 18
```

```
if edad >= 18:
```

```
    print("Eres mayor de edad.")
```

```
# En este caso, como la condición (18 >= 18) es True,
```

```
# el mensaje "Eres mayor de edad." se imprimirá.
```

1.2 Sentencia if-else

Con if-else, puedes definir un camino alternativo. Si la condición del if es True, se ejecuta su bloque de código; si es False, se ejecuta el bloque de código del else. Solo se ejecutará uno de los dos.

Ejemplo:

```
temperatura = 25
```

```
if temperatura > 30:
    print("Hace demasiado calor, ¡enciende el aire acondicionado!")
else:
    print("La temperatura es agradable.")

# Como 25 no es mayor que 30, la condición es False,
# por lo que se ejecutará el bloque del 'else'.
```

1.3 Sentencia if-elif-else

Esta estructura es ideal para encadenar múltiples condiciones. Python evaluará las condiciones en orden, de arriba abajo. En cuanto encuentre una que sea True, ejecutará el bloque de código correspondiente y saldrá de toda la estructura if-elif-else. Si ninguna de las condiciones es True, se ejecutará el bloque else (si existe).

Ejemplo:

```
puntuacion = 85

if puntuacion >= 90:
    print("Excelente, has obtenido una A.")
elif puntuacion >= 80:
    print("Buen trabajo, has obtenido una B.")
elif puntuacion >= 70:
    print("Puedes mejorar, has obtenido una C.")
else:
    print("Sigue practicando, has suspendido.")

# La condición de 'puntuacion >= 90' es False.
# La siguiente, 'puntuacion >= 80', es True, por lo que
# se ejecutará el bloque y el programa continuará.
```

1.4 Operadores lógicos y de comparación

Las condiciones se construyen utilizando operadores de comparación (==, !=, <, >, <=, >=) y operadores lógicos (and, or, not). Estos operadores te permiten combinar y negar condiciones para crear lógicas más complejas.

Ejemplo:

```
es_estudiante = True
```

```
edad = 20
```

```
# Uso de 'and' para que ambas condiciones se cumplan
```

```
if es_estudiante and edad >= 18:
```

```
    print("Cumple los requisitos para un descuento de estudiante mayor de edad.")
```

```
# Uso de 'or' para que al menos una condición se cumpla
```

```
if es_estudiante or edad >= 65:
```

```
    print("Cumple los requisitos para un descuento.")
```

```
# Uso de 'not' para negar una condición
```

```
if not es_estudiante:
```

```
    print("No es un estudiante.")
```

1.5 Sentencia match-case

A partir de Python 3.10, se incorporó una nueva estructura de control llamada match-case, que permite comparar un valor con varios patrones posibles, de manera similar a un switch-case en otros lenguajes de programación.

Esta sentencia facilita la lectura del código cuando existen múltiples condiciones que dependen del valor de una misma variable.

Ejemplo:

```
opcion = "b"
```

match opcion:

case "a":

print("Has elegido la opción A")

case "b":

print("Has elegido la opción B")

case "c":

print("Has elegido la opción C")

case _:

print("Opción no válida")

Explicación:

- match evalúa el valor de la variable (opcion).
- Cada case representa una posible coincidencia.
- El guion bajo (_) actúa como un caso **por defecto**, equivalente a else en una estructura condicional.

Ejemplo con condiciones más avanzadas:

numero = 15

match numero:

case n if n < 0:

print("El número es negativo")

case n if 0 <= n < 10:

print("El número es menor que 10")

case n if 10 <= n < 20:

print("El número está entre 10 y 19")

case _:

print("El número es 20 o mayor")

En este ejemplo, se combinan patrones con **condiciones (if)** para evaluar rangos de valores.

Ventajas de match-case:

- Mejora la legibilidad cuando hay muchas comparaciones sobre una misma variable.
- Permite realizar **patrones complejos**, como coincidencias de tipos o estructuras de datos.
- Sustituye a las largas cadenas de if-elif-else en ciertos casos.

2. Estructuras de repetición (Bucles)

Los bucles te permiten ejecutar un bloque de código de forma repetida. Esto es útil para procesar listas, buscar elementos o realizar tareas que se repiten un número determinado de veces.

2.1 Bucle while

El bucle while ejecuta su bloque de código **mientras** la condición que se evalúa sea True. Es crucial asegurarse de que la condición, en algún momento, se vuelva False para evitar un bucle infinito.

Ejemplo:

```
contador = 0
```

```
while contador < 5:
```

```
    print(f"El contador es: {contador}")
```

```
    contador += 1 # Incrementamos el contador en cada iteración
```

```
# Este bucle se ejecutará 5 veces, mientras que 'contador' sea menor que 5.
```

2.2 Bucle for

El bucle for está diseñado para iterar sobre los elementos de una secuencia (como una lista, una cadena, un rango o una tupla). En cada iteración, una variable tomará el valor del siguiente elemento de la secuencia.

Ejemplo:

```
# Iterar sobre una lista
```

```
frutas = ["manzana", "banana", "cereza"]
```

```
for fruta in frutas:
```

```
    print(f"Me gusta la {fruta}.")
```

Ejemplo:

```
# Iterar sobre una cadena de texto
```

```
palabra = "Python"
```

```
for letra in palabra:
```

```
    print(f"Letra: {letra}")
```

Ejemplo:

```
# Iterar sobre un rango de números
```

```
# La función range(5) genera los números 0, 1, 2, 3, 4
```

```
for numero in range(5):
```

```
    print(f"Número: {numero}")
```


3. Estructuras de salto en bucles

A veces, necesitas modificar el comportamiento normal de un bucle, como salir de él prematuramente o saltar una iteración específica. Para ello, Python ofrece tres sentencias clave.

3.1 Sentencia break

La sentencia break **interrumpe por completo el bucle** en el que se encuentra. El programa salta al primer bloque de código que esté justo después del bucle.

```
for i in range(10):  
    if i == 5:  
        print("¡He encontrado el 5! Saliendo del bucle.")  
        break # El bucle se detiene aquí  
    print(i)  
# El programa imprimirá 0, 1, 2, 3, 4 y luego el mensaje,  
# sin llegar a imprimir el 5, 6, 7, etc.
```

3.2 Sentencia continue

La sentencia continue **salta a la siguiente iteración del bucle**. El código que esté después de continue dentro del bucle no se ejecutará en esa iteración específica.

```
for i in range(5):  
    if i == 2:  
        print("Saltando la iteración 2...")  
        continue # Salta directamente a la siguiente iteración (i = 3)  
    print(f"Valor actual: {i}")  
  
# El programa imprimirá 0, 1, el mensaje, 3, 4.  
# La línea 'print(f"Valor actual: {i}")' no se ejecutará cuando i sea 2.
```

3.3 Sentencia pass

La sentencia pass es una operación nula. No hace nada. Se utiliza como **marcador de posición** cuando la sintaxis de Python requiere una sentencia, pero no quieres que se ejecute ninguna acción. Es común en la fase de desarrollo para indicar que un bloque de código se implementará más tarde.

```
for i in range(3):  
    # En el futuro, aquí implementaré una lógica compleja  
    pass
```

```
def funcion_futura():  
    # Esta función está definida pero aún no hace nada  
    pass
```

4. Control de excepciones

Las excepciones son errores que ocurren durante la ejecución de un programa. Si no se gestionan, pueden provocar que el programa se detenga abruptamente. El control de excepciones te permite manejar estos errores de forma elegante y evitar que tu aplicación se cierre.

4.1 ¿Qué es una excepción?

Una **excepción** es un evento que interrumpe el flujo normal de un programa. Por ejemplo, intentar dividir un número por cero o acceder a un índice que no existe en una lista.

4.2 Manejo de excepciones con try-except

El bloque try contiene el código que podría generar una excepción. Si ocurre una excepción en ese bloque, la ejecución se detiene y el control se transfiere al bloque except correspondiente, que contiene el código para manejar ese error.

try:

```
numero1 = 10

numero2 = int(input("Introduce un número para dividir 10: "))

resultado = numero1 / numero2

print(f"El resultado es: {resultado}")
```

except ZeroDivisionError:

```
print("Error: No se puede dividir por cero.")
```

except ValueError:

```
print("Error: Debes introducir un número entero válido.")
```

Si el usuario introduce '0', se activa el 'ZeroDivisionError'.

Si el usuario introduce 'abc', se activa el 'ValueError'.

4.3 Bloques else y finally

Puedes añadir bloques adicionales para un manejo más completo:

- **else:** Su código se ejecuta **solo si no ocurre ninguna excepción** en el bloque try.
- **finally:** Su código se ejecuta **siempre**, tanto si hubo una excepción como si no. Es ideal para tareas de limpieza, como cerrar un archivo.

try:

```
archivo = open("datos.txt", "r")
```

```
linea = archivo.readline()
```

```
print("Contenido leído:", linea)
```

except FileNotFoundError:

```
print("Error: El archivo no se encontró.")
```

else:

```
print("La lectura del archivo fue exitosa.")
```

finally:

```
# Este bloque se ejecuta siempre, asegurando que el archivo se cierre.
```

```
if 'archivo' in locals():
```

```
    archivo.close()
```

```
print("Fin del manejo de archivos.")
```

4.4 Lanzar excepciones con raise

La palabra clave raise se utiliza para **generar una excepción de forma explícita**. Esto permite indicar que ha ocurrido un error o que una condición no permitida se ha cumplido, interrumpiendo el flujo normal del programa.

Sintaxis:

```
raise TipoDeError("Mensaje de error")
```

- TipoDeError: clase de la excepción (por ejemplo, ValueError, TypeError, ZeroDivisionError, etc.).
- "Mensaje de error": texto que describe la causa del error.

Ejemplo:

```
edad = -5
```

```
if edad < 0:
```

```
    raise ValueError("La edad no puede ser negativa")
```

Si se ejecuta este código, Python interrumpe el programa y muestra:

ValueError: La edad no puede ser negativa

4.5 Creación de excepciones personalizadas

Puedes definir tus propias excepciones heredando de la clase base Exception. Esto es útil para crear errores específicos que sean relevantes para tu aplicación.

```
class SaldoInsuficiente(Exception):
```

```
    """Excepción personalizada para cuando el saldo de una cuenta no es suficiente."""
```

```
    pass
```

```
def retirar_dinero(saldo, cantidad):
```

```
    if cantidad > saldo:
```

```
        raise SaldoInsuficiente("No hay suficiente saldo para completar la operación.")
```

```
    return saldo - cantidad
```

```
try:
```

```
    nuevo_saldo = retirar_dinero(100, 150)
```

```
    print("Retiro exitoso. Nuevo saldo:", nuevo_saldo)
```

```
except SaldoInsuficiente as e:
```

```
    print(f"Error en la operación: {e}")
```

5. Aserciones

Las aserciones (assert) son una herramienta para **verificar condiciones internas** en tu código que deberían ser True en un estado de funcionamiento normal. Se usan principalmente para la depuración y para documentar supuestos sobre el estado del programa.

5.1 ¿Qué es una aserción?

Una aserción es una declaración que se supone que siempre debe ser True. Si la condición es False, Python lanza una excepción del tipo AssertionError, lo que detiene la ejecución del programa.

```
def calcular_promedio(lista_numeros):  
    assert len(lista_numeros) > 0, "La lista no puede estar vacía"  
    return sum(lista_numeros) / len(lista_numeros)  
  
# Con una lista vacía, el programa lanzará un AssertionError  
# calcular_promedio([])  
  
# Con una lista válida, la aserción se ignora  
print(calcular_promedio([10, 20, 30]))
```

6. Prueba, depuración y documentación de la aplicación

Para desarrollar software de calidad, no basta solo con escribir código. Es esencial probarlo, corregir errores y documentarlo para que sea comprensible.

6.1 Pruebas de programas

Probar un programa implica ejecutar tu código bajo diferentes condiciones para asegurarte de que se comporta como esperas. Hay diferentes niveles de prueba (unitarias, de integración, etc.), pero el objetivo principal es validar que la lógica es correcta y que no hay fallos.

6.2 Depuración de código

La **depuración** es el proceso de encontrar y corregir errores o *bugs*. Las herramientas y técnicas más comunes incluyen:

- **Impresiones de seguimiento (print):** Una forma sencilla de ver el valor de las variables en diferentes puntos de tu código.
- **Depuradores integrados:** Los entornos de desarrollo (IDE) como VS Code, PyCharm o incluso Jupyter Notebooks tienen depuradores que te permiten ejecutar el código paso a paso, inspeccionar variables y seguir el flujo de ejecución.
- **Módulo pdb:** El depurador estándar de Python, que te permite interactuar directamente con tu programa en la línea de comandos.

6.3 Documentación del código

Una buena documentación hace que tu código sea más fácil de entender y mantener, tanto para otros programadores como para tu yo del futuro.

- **Comentarios:** Usa comentarios para explicar por qué haces algo, especialmente en partes complejas del código.
- **Docstrings:** Son cadenas de texto que se colocan al inicio de módulos, clases y funciones. Se usan para explicar su propósito, sus parámetros y lo que devuelven.

```
def calcular_area_rectangulo(base, altura):
```

```
    """
```

```
    Calcula el área de un rectángulo.
```

Args:

base (float): La longitud de la base del rectángulo.

altura (float): La altura del rectángulo.

Returns:

float: El área total del rectángulo.

"""

return base * altura

La documentación se puede acceder con help()

help(calcular_area_rectangulo)